

Micro-services Labs 2: Polymove

Introduction

In the previous lab, you implemented synchronous REST communication between services. In this lab, you will discover a different communication pattern: **gRPC** (Remote Procedure Call).

You will also introduce a new service, **MI8**, which acts as a news intelligence agency, and implement **Colporteurs** — scripts that feed news into the system.

Learning Objectives

By the end of this lab, you will have:

- Implemented a gRPC server exposing news data
- Implemented a gRPC client consuming that data
- Used Redis as a fast key-value store
- Built a news ingestion pipeline with Colporteurs
- Implemented a real-time city scoring system based on news sentiment

Part 1: let's start by defining our new service: MI8

MI8 is the intelligence service of Polymove. It collects, stores, and exposes news about cities around the world. Other services query MI8 to get information about destinations.

Objective

Implement a GRPC serveur on the MI8 service to expose news.

Consumes the news in a newly added GRPC Client in the Polytech Service.

Resources

- [gRPC Documentation](#)
- [Protocol Buffers Language Guide](#)

Spec:

- Communicate via gRPC to Polytech
- Save news to Redis

Data Model of news

Name

Source

Date

Tags

City

Country

Endpoint to add in MI8 (gRPC)

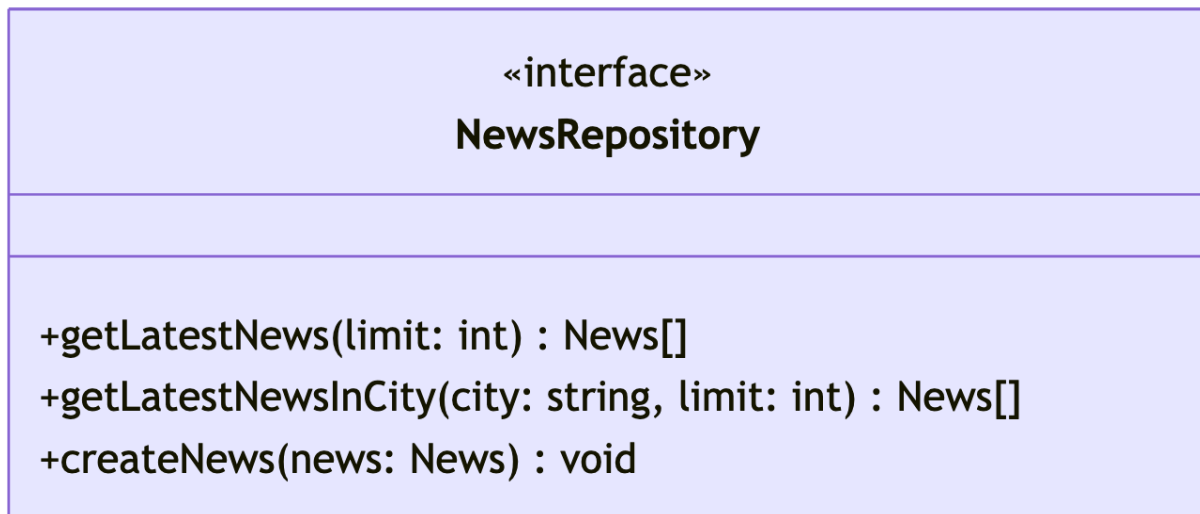
Method	Description
<code>GetLatestNews(limit: int): News[]</code>	Retrieve the latest <limit> news
<code>GetLatestNewsInCity(city: string, limit: int): News[]</code>	Retrieve the latest news in a city

Implementation steps in MI8

Define the Protobuf File Create a `.proto` file that defines: the message and the service with the methods

Generate Language-Specific Code Compile your `.proto` file to generate the classes/structs for your chosen language.

Implement the Repository Layer You will use the Adapter Pattern to abstract data storage:



Define an `ArrayNewsRepository` service that implement the `NewsRepository` interface.

Implement the gRPC Server Create the gRPC server that implements the service methods and uses the repository to fetch/store data

Populate Test Data: Add a few hardcoded news items in your `ArrayNewsRepository` to test your server.

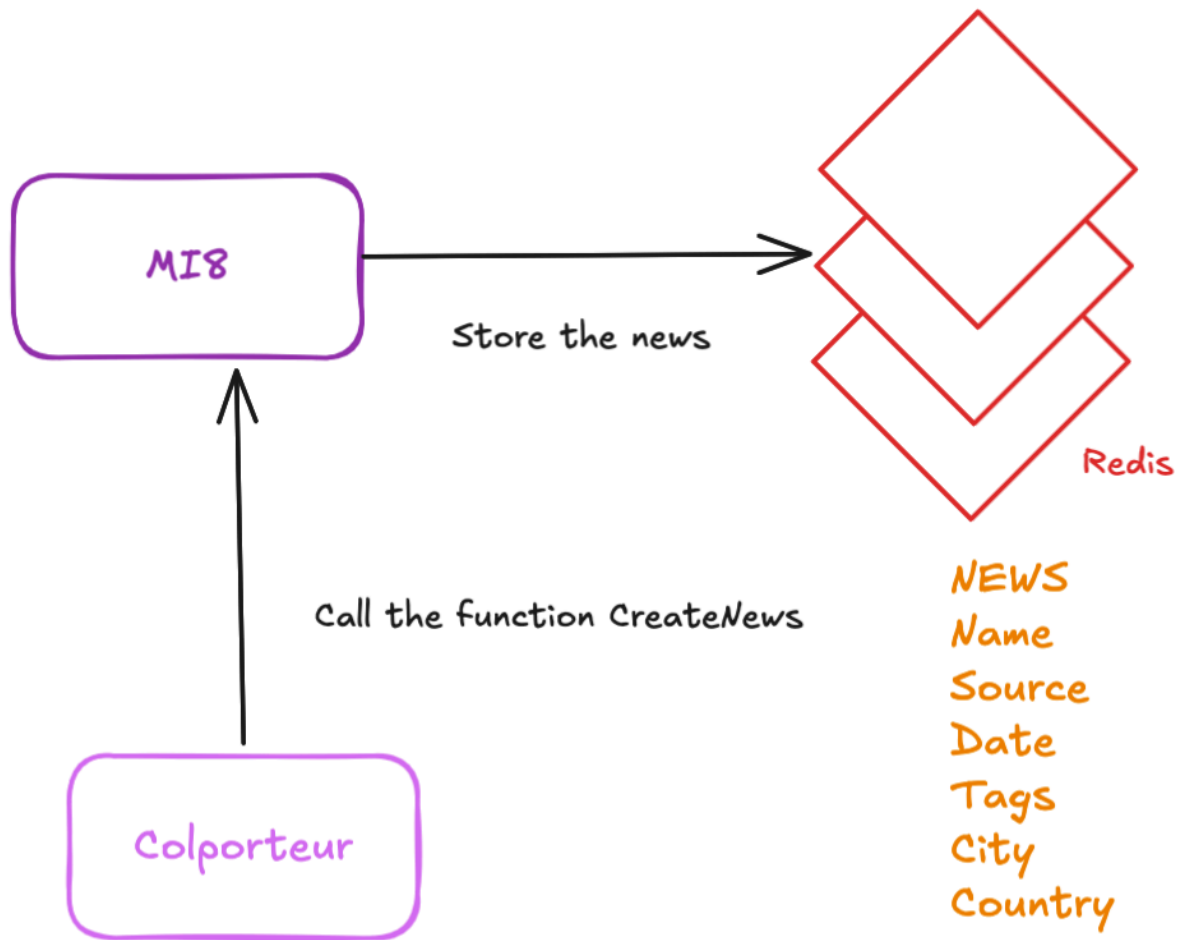
Implementation steps in Polytech

1. Copy or share the `.proto` file with the Polytech service
2. Generate the client code for your language
3. Implement a client class that connects to MI8
4. Test the connection by calling `GetLatestNews` and logging the results

Part 2: Implement Colporteurs & Redis Storage

Problem

Our system isn't full complete because news aren't store in a DB. If the service crash, we lose data. Also, we actually don't listen for news from Colporteurs. You will implement that



Objective

1. Switch from in-memory storage to **Redis**
2. Implement the **CreateNews** gRPC method
3. Create **Colporteur** scripts that feed news into MI8

Ressources

- [Redis Commands](#)
- [Redis Sorted Sets](#)

Endpoint to add in MI8 (gRPC)

Function	Description
CreateNews(news : News)	Create a news and save it

Implementation steps in MI8

Add the method createNews in your abstract class related to News.

Implement the RedisNewsRepository that perform operation with Redis. For Redis, see online what is the best way to store informations. Consider these questions for your Redis data structure:

- How will you store individual news items?
- How will you efficiently retrieve the latest N news?
- How will you filter by city?

Colporteur

A Colporteur is a standalone script that connects to MI8 via gRPC and sends news items.

Create at least one Colporteur that sends sample news to MI8.

Tests

Query news from Polytech to ensure MI8 retrieve news from Redis. Use Inspector tools for Redis if errors and log debugging

Part 3: City scoring system

Context

Polytech wants to help students choose their internship destination. To do this, they need a **score for each city** based on the news sentiment. Cities with positive news should rank higher.

Objective

Implement a scoring system that updates city scores in real-time as news arrives.

Data Model: CityScore

- City
- Country
- Scores:
 - QualityOfLife (int)
 - Safety
 - Economy
 - Culture
- LastUpdated (date)

New methods for MI8

Function	Description
<code>GetCityScore(city: string): CityScore</code>	Return the scores of one city
<code>GetTopCities(limit: int): CityScore[]</code>	Return the top <limit> cities, ordered by ascending score

Scoring Algorithm

Each news item affects the city's different scores based on its tags. Implement the following scoring rules:

- Each city starts with a base score of **1000**
- When news is created, calculate the score impact from all its tags
- Update the city's scores accordingly
- A city's score cannot go below **0**

Here are the tags and their corresponding changes reflected to the scores:

Tag	Safety	Economy	Quality of Life	Culture
innovation	+20	+60	+30	+5
culture		+15	+40	+75

healthcare	+30	+20	+30	
entertainment		+20	+25	+35
crisis	-80	-100	-60	-30
crime	-120	-50	-80	-40
disaster	-100	-70	-90	-30

Example

News: "Paris Gentle Mattes won's the Call Of Duty Major in Dallas"

City: Berlin

Tags: ["culture", "entertainment",]

Scores calculation:

economy: +35 (15+20)

quality of life: +65 (40+25)

culture: +110 (35+75)

Dalla's score:

Economy: 1000 → 1035

QoL: 1000 → 1065

Culture: 1000 → 1110

Deliverable

Once completed, commit your work to a branch named:

lab-mi8-grpc-serveur

Be sure to put your repository in public or invite me with my [GridexX](#)